

Preguntas recopiladas

Recopilación de preguntas realizadas en el examen teórico de promoción de Ingeniería de Software 2 entre 2024 y 2026.

- Las / en una misma pregunta representan formas distintas de preguntar lo mismo.
 - Si el enunciado solo pide mencionar no es necesario desarrollar.
-

1. Exprese 4 características de un buen SRS / Desarrollar 4 características de SRS.

- Correcto: Un SRS es correcto si cada requisito declarado se encuentra en el software.
- No ambiguo: Un SRS es inequívoco si y sólo si cada requisito declarado tiene solo una interpretación.
- Completo: Reconoce todos los requisitos externos impuestos por especificaciones del sistema.
- Trazabilidad: Claridad del origen de cada requerimiento y su trazabilidad hacia los requerimientos futuros.
- Ambiente del SRS: el software puede contener toda la funcionalidad del proyecto o puede ser parte de un sistema más grande.
- Consistente: la consistencia se refiere a la consistencia interior. Si un SRS no está de acuerdo con algún documento del nivel superior entonces no es consistente.
- Priorizado: un SRS es priorizado por la importancia de sus requerimientos particulares.

2. Explique 4 diferencias entre UX (experiencia de usuario) y UI (desarrollo de interface).

3. Indique y describa 3 principios aplicados al auditor informático.

- **Principio de Beneficio del Auditado:** El auditor tiene la obligación de buscar la **máxima eficacia y rentabilidad** de los medios informáticos de la empresa que está siendo auditada. Bajo este principio, el profesional

debe priorizar el éxito del cliente y **no debe obtener beneficios propios** o personales a partir de la auditoría.

- **Principio de Calidad:** Este principio establece que el auditor debe prestar sus servicios haciendo uso de todas las posibilidades de la **ciencia y los medios técnicos** que tenga a su alcance. Además, debe contar con **absoluta libertad** para utilizar dichos medios en las condiciones técnicas que considere adecuadas para cumplir con su labor de forma óptima.
- **Principio de Independencia:** Es fundamental que el auditor exija y mantenga una **total autonomía e independencia** en su trabajo. Este principio se aplica sin importar si el auditor es un profesional externo contratado o si tiene una relación de dependencia laboral directa con la empresa auditada.

4. Explique los 4 tipos de diseño / Mencione los 4 tipos de diseño.

- **Diseño de datos:** Transforma el modelo de dominio en estructuras de datos, objetos y relaciones específicas que el software manejará (Objetos1).
- **Diseño arquitectónico:** Define la estructura global del sistema, identificando los subsistemas y estableciendo los patrones de diseño y marcos de control (Objetos2).
- **Diseño a nivel de componentes:** Traduce los elementos estructurales en una descripción procedimental detallada de cada componente del software (DTE).
- **Diseño de interfaz:** Describe cómo se comunica el sistema internamente, con otros sistemas externos y con los usuarios humanos (Inge2).

5. Explique 4 tipos de rejuvenecimiento de software / Mencione 4 estrategias de rejuvenecimiento de software / Menciona las 4 estrategias de rejuvenecimiento de software y explique una.

- **Re-estructuración:** Se reorganiza el código para hacerlo más comprensible y fácil de cambiar, buscando reducir su complejidad ciclomática sin alterar su función.
- **Re-documentación:** Proceso de análisis estático del código fuente para generar documentación técnica actualizada (diagramas, flujos de datos,

etc.) que se ha perdido o no existe.

- **Ingeniería inversa:** Analiza un sistema existente para recuperar y entender sus abstracciones de diseño y especificación partiendo únicamente del código fuente.
- **Re-ingeniería:** Extensión de la ingeniería inversa que no solo recupera el diseño, sino que genera un nuevo código fuente con mejor calidad y estructura.

14. Defina 4 criterios técnicos para un buen diseño de software / Describa 4 pautas de criterios técnicos para un buen diseño de software / Desarrollar 4 características del diseño de software.

- **Independencia funcional:** El diseño debe dar lugar a componentes cuyas funciones sean lo más independientes posible entre sí, buscando **alta cohesión y bajo acoplamiento**.
- **Modularidad:** El software debe estar dividido en componentes (módulos) abordados por separado que se integran para satisfacer los requisitos.
- **Estructura arquitectónica sólida:** Debe basarse en **patrones de diseño reconocibles** y permitir una implementación evolutiva.
- **Interfaces simplificadas:** Debe conducir a conexiones que **reduzcan la complejidad** entre los módulos y con el entorno externo.
- **Representación y legibilidad:** Debe ser una **guía legible** para los constructores y proporcionar una visión completa (completitud) del sistema.

15. Explique en qué consiste hacer el análisis y planeación de riesgos.

1. Análisis de Riesgos

Consiste en evaluar cada riesgo identificado para determinar su importancia y establecer un **listado de priorización**.

- **Componentes de evaluación:** Cada riesgo se califica bajo dos criterios:
 - **Probabilidad:** La estimación de qué tan posible es que el evento ocurra, utilizando escalas que van desde "Bastante improbable" (<10%) hasta "Bastante probable" (>75%).

- **Impacto:** La magnitud de la pérdida o daño que causaría, clasificado como Catastrófico (cancelación del proyecto), Serio, Tolerable o Insignificante.
- **Construcción de la Tabla de Riesgos:** Se crea una tabla con el nombre del riesgo, su categoría, probabilidad e impacto.
- **Línea de Corte:** La lista se ordena por importancia y se traza una línea de corte. Los riesgos por **encima de la línea** (generalmente los de mayor probabilidad y/o impacto) son los que recibirán atención inmediata, mientras que los que están por debajo se consideran de segundo orden.

2. Planeación de Riesgos

En esta etapa, se consideran los riesgos que quedaron por encima de la línea de corte para definir las estrategias de tratamiento.

Existen tres estrategias principales para abordar los riesgos aceptados:

- **Evitar el riesgo:** Se rediseña el sistema o el plan para que el evento no pueda ocurrir.
- **Minimizar el riesgo (Mitigación):** Se aplican acciones proactivas para **reducir la probabilidad** de que el riesgo se presente.
- **Plan de contingencia:** Se acepta que el riesgo puede aparecer y se define **qué hacer cuando ocurra** para minimizar sus consecuencias negativas.

Para decidir qué tratamiento seguir, también se puede calcular la **Exposición al Riesgo (E)**, que es el producto de la probabilidad (P) por el costo del impacto (C) ($E = P \times C$). Esto permite a la gerencia evaluar si el costo de aplicar una estrategia es proporcional al daño potencial.

16. Mencione 2 características y 2 desventajas de los sistemas distribuidos.
17. Describir 2 ventajas y 2 desventajas del patrón arquitectónico de "Repositorio".

Una de las ventajas es la eficiencia al compartir una gran cantidad de datos. No se transmiten de un subsistema a otro porque se comunican mediante el modelo de datos central.

Otra ventaja es que los subsistemas cuentan con un bajo acoplamiento, lo que se representa con independencia funcional.

Una de las desventajas críticas del patrón de repositorio es que todos los subsistemas deben estar acordes al modelo de datos central. Esto puede afectar al rendimiento.

La evolución puede ser compleja a medida que crece el volumen de datos. Esto puede dificultar la migración, o incluso puede resultar imposible de realizar.

Una última desventaja es que los subsistemas pueden contar con distintos tipos de protección o políticas de seguridad, pero el modelo de repositorio impone las mismas para todos los subsistemas.

18. Explique la diferencia entre pruebas top down y botton down.

La diferencia principal entre las pruebas **top-down** (descendentes) y **bottom-up** (ascendentes) radica en la dirección en la que se integran y prueban los módulos dentro de la estructura jerárquica del software. Ambas son estrategias de **pruebas de integración**.

A continuación se detallan sus diferencias clave:

1. Integración Top-Down (Descendente)

- **Dirección:** Los módulos se integran moviéndose **hacia abajo por la jerarquía de control**, comenzando por el módulo o programa principal.
- **Punto de partida:** El módulo de control principal se utiliza como el primer "conductor" de la prueba.
- **Dependencias (Resguardos/Stubs):** Como los módulos de niveles inferiores aún no están desarrollados o integrados, se deben utilizar **resguardos (stubs)** para sustituirlos y simular su comportamiento.
- **Ventaja:** Permite verificar las funciones de control de alto nivel de forma temprana en el proyecto.
- **Desventaja:** La principal desventaja es la necesidad de escribir y mantener los **resguardos**, lo que genera una sobrecarga de trabajo adicional.

2. Integración Bottom-Up (Ascendente)

- **Dirección:** Comienza la construcción y las pruebas con los **módulos atómicos** (los de nivel más bajo en la estructura) y progresa hacia arriba.

- **Punto de partida:** Módulos de bajo nivel que realizan funciones específicas.
- **Dependencias (Controladores/Drivers):** Como los módulos superiores aún no están integrados, se requieren **controladores (drivers)** para coordinar la entrada y salida de datos hacia los módulos que se están probando.
- **Ventaja:** No se necesitan resguardos (stubs), ya que al integrar de abajo hacia arriba, los módulos subordinados siempre están disponibles.
- **Desventaja:** El **programa como entidad completa no existe** hasta que se ha agregado y probado el último módulo (el principal).

19. Explique la diferencia entre auditor y perito técnico.

El auditor cuenta con acceso al sistema interno, posee un enfoque estratégico y técnico, y tiene como objetivo identificar riesgos, vulnerabilidades y verificar controles, mientras que el perito se encarga de aspectos legales. Tiene un enfoque forense y legal. Recopila pruebas digitales y analiza incidentes de seguridad.

20. Explique la diferencia entre pruebas alfa y beta.

Las pruebas **alfa** y **beta** son las dos categorías principales de las **pruebas de aceptación**, las cuales son realizadas por el usuario final en lugar del equipo de desarrollo.

Las diferencias fundamentales entre ambas se detallan a continuación:

1. Ubicación y Participantes

- **Pruebas Alfa:** Se llevan a cabo en el **lugar de desarrollo**. El cliente utiliza el software mientras el **desarrollador está presente** como observador, registrando directamente los errores y problemas de uso detectados.
- **Pruebas Beta:** Se realizan en los **lugares de trabajo de los clientes** o usuarios finales. En este caso, el **desarrollador no está presente**; es el cliente quien debe registrar los problemas que encuentre e informar al desarrollador a intervalos regulares.

2. Entorno y Control

- **Pruebas Alfa:** Se desarrollan bajo un **entorno controlado** por la organización que crea el software.
- **Pruebas Beta:** Representan una aplicación "en vivo" del software en un **entorno que no es controlado**.

3. Etapa y Finalidad

- **Pruebas Alfa:** Se ejecutan después de completar las pruebas de unidad, integración y sistema, pero **no sobre la versión final** del producto, ya que todavía podrían añadirse funcionalidades.
- **Pruebas Beta:** Se consideran la **última fase de las pruebas** antes del lanzamiento definitivo. Utilizan técnicas de **caja negra** para validar el comportamiento global. En ocasiones, la versión beta se lanza al mercado para obtener retroalimentación masiva antes de publicar la versión final.

21. Explique 4 principios de Nielsen / Mencione 4 principios de Nielsen.

- **Feedback (Visibilidad del estado):** El sistema siempre debe mantener informado al usuario sobre lo que está ocurriendo mediante respuestas gráficas o textuales oportunas.
- **Consistencia:** Debe existir uniformidad visual y terminológica para que el usuario no tenga dudas sobre el comportamiento del sistema en diferentes contextos.
- **Prevención de errores:** Es mejor diseñar una interfaz que evite que el error ocurra (usando valores por defecto o rangos) que mostrar un mensaje de error después.
- **Salidas evidentes:** El usuario debe tener siempre el control para deshacer, rehacer o salir de un estado o pantalla de forma sencilla e identificable.

22. Explicar como aplicar algún principio de Nielsen en una app de reserva de turnos.

Feedback: en una app de reserva de turnos, al momento de reprogramar un turno:

Confirmación de la acción: Una vez seleccionado el nuevo horario, la app debe mostrar un mensaje explícito: "Su turno ha sido reprogramado correctamente para el día [Fecha] a las [Hora]".

Información de restricción de estado: Para cumplir con el requerimiento de indicar que no es posible volver a reprogramar, el feedback debe incluir una aclaración de las reglas de negocio aplicadas al nuevo estado: "Nota: Este turno ha sido reprogramado por única vez. No se permiten cambios adicionales sobre esta nueva reserva".

23. ¿Qué es una "línea base" en la Gestión de la Configuración del Software (GCS) y cuál es su importancia según el estándar IEEE?

Una línea base es una especificación o producto que se revisó formalmente y se llegó a un acuerdo. Sirve como punto de referencia para gestionar cambios posteriores.

24. ¿Qué mide la métrica de Punto Función (PF) y en qué se diferencia de la métrica de Líneas de Código (LDC)?

25. Indica funciones del auditor informático.

El auditor informático, a diferencia del consultor, cuenta con acceso al sistema interno por lo que tiene un enfoque más específico y técnico destinado a identificar riesgos, vulnerabilidades y verificar controles.

26. Imagina que estás a cargo de un proyecto de software para una aplicación móvil bancaria. Utilizando el concepto de Gestión de Riesgos, identifica dos eventos no deseados (riesgos) que podrían amenazar el éxito del proyecto y propone una breve estrategia de mitigación para cada uno.

Riesgo N°1: cambios en el panorama económico. Estrategia de mitigación: monitorear indicadores económicos.

Riesgo N°2: fallos en la seguridad del producto. Estrategia de mitigación: aplicar buenas prácticas de seguridad desde el inicio.

27. ¿Cuáles son las características del mantenimiento de software?

1. Características Técnicas y del Proceso

- **Reinicio del ciclo de vida:** Las tareas de mantenimiento generalmente provocan que se deban **reiniciar las fases de análisis, diseño e implementación**. Esto es necesario para comprender el alcance del cambio, rediseñar para incorporar las modificaciones y recodificar.
- **Efectos secundarios:** Existe el riesgo de que las modificaciones generen efectos no deseados sobre el **código fuente, los datos o la documentación** existente.

- **Dificultad de comprensión:** Resulta problemático y complejo comprender **código ajeno**, especialmente cuando la documentación es inadecuada, inexistente o el sistema carece de una estructura modular.
- **Limitaciones del diseño:** No siempre se prevén los cambios futuros durante la etapa de diseño inicial, lo que dificulta las intervenciones posteriores.

2. Aspectos Económicos y Operativos

- **Alto costo relativo:** El mantenimiento representa una carga económica significativa, involucrando entre un **40% y un 70% del costo total** del desarrollo de software.
- **Disminución de otros desarrollos:** Una consecuencia directa de dedicar recursos al mantenimiento es la reducción de la capacidad de la organización para iniciar o avanzar en nuevos proyectos.
- **Insatisfacción del cliente:** La aparición de errores durante esta fase es una fuente directa de insatisfacción para los usuarios finales.
- **Percepción del trabajo:** Generalmente, para los profesionales del software, el mantenimiento no se percibe como una tarea motivadora o "atractiva" en comparación con el desarrollo de sistemas nuevos.

3. Dinámica de Evolución (Leyes de Lehman)

El mantenimiento sigue pautas de comportamiento descritas por las **Leyes de Lehman**, que incluyen:

- **Cambio continuado:** Un programa que se usa en un entorno real debe cambiar necesariamente o se volverá progresivamente menos útil.
- **Complejidad creciente:** A medida que el software evoluciona, su estructura tiende a ser más compleja, requiriendo recursos extras para simplificarla y preservarla.
- **Decremento de la calidad:** La calidad del sistema disminuirá a menos que se adapte rigurosamente a los cambios de su entorno de funcionamiento.

4. Roles y Participantes

- **Equipo independiente:** A menudo, el equipo que mantiene el sistema no es el mismo que lo desarrolló.

- **Involucramiento de usuarios:** El equipo de mantenimiento debe trabajar estrechamente con usuarios y operadores, quienes reportan problemas o sugieren mejoras necesarias para que el software siga siendo una herramienta efectiva en su entorno.

28. Explicar técnica de Planning Póker.

La técnica de Planning Poker es una herramienta de estimación colaborativa basada en el consenso, utilizada principalmente en el desarrollo de software ágil (como el marco de trabajo Scrum) para predecir el esfuerzo requerido para completar historias de usuario o elementos del backlog.

1. Propósito y Participantes

El objetivo principal es obtener estimaciones precisas fomentando la **participación y el entendimiento compartido** de todo el equipo. En esta técnica participan **todas las personas comprometidas en el desarrollo** del software.

2. Elementos: Las Cartas de Fibonacci

Se denomina "póker" porque los miembros del equipo utilizan **cartas numeradas** para realizar sus estimaciones. Generalmente, se utiliza la **serie de Fibonacci** (por ejemplo: 1, 2, 3, 5, 8, 13, 21) para representar el esfuerzo, ya que esta escala ayuda a reflejar la incertidumbre inherente a tareas más grandes.

3. Pasos del Proceso (Iterativo)

De acuerdo con la documentación, el proceso sigue estos pasos estructurados:

1. **Reunión del equipo:** Los integrantes se sientan alrededor de una mesa y cada uno recibe un juego de cartas.
2. **Presentación de la historia:** El dueño del producto (*Product Owner*) lee una historia de usuario al equipo.
3. **Aclaración:** Se realiza una ronda de preguntas para resolver dudas y entender bien el alcance de la historia.
4. **Selección privada:** Cada miembro selecciona una carta en privado que represente su estimación del esfuerzo.
5. **Revelación y discusión:** Todos muestran sus cartas al mismo tiempo. Si no hay unanimidad, se discuten las razones de las estimaciones más

altas y más bajas para llegar a un acuerdo. El proceso de votación se repite hasta alcanzar el **consenso**.

6. **Repetición:** Se continúa con el mismo flujo para el resto de las historias de usuario del Sprint o el backlog.

4. ¿Cuándo se debe utilizar?

Esta técnica es ideal cuando:

- Se utilizan **metodologías ágiles** para estimar historias de usuario.
- Se busca que el equipo sea **dueño de sus propias estimaciones**, lo que genera mayor compromiso.
- Se desea identificar suposiciones, dependencias y riesgos de forma proactiva a través de la discusión abierta.
- Las tareas varían en complejidad y requieren ser bien entendidas por todos antes de empezar.

29. Explicar las 4 P de la Gestión de Proyectos / Nombrar las 4P y marcar la más importante.

La administración de un proyecto de software debe enfocarse en cuatro áreas críticas para tener éxito:

- **Personal (RRHH):** Es el elemento más importante y crítico del proyecto. Representa el capital intelectual y es quien realiza el esfuerzo.
- **Producto:** Se refiere al resultado intangible (el software) que se debe construir. Es fundamental realizar una buena planificación para definir soluciones alternativas y medir el progreso.
- **Proceso:** Proporciona el marco de trabajo (las tareas y actividades) desde el cual se establece el plan detallado para el desarrollo.
- **Proyecto:** Es el esfuerzo temporal para crear el producto único. Debe ser planeado y controlado para manejar la complejidad del desarrollo.

30. Explicar las pruebas de límites (caja negra).

Técnicamente conocida como **Análisis de Valores Límite (AVL)**, es una técnica que complementa a la partición equivalente dentro de las pruebas de **caja negra**.

- **Concepto:** Se basa en el principio de que los errores tienden a ocurrir con mayor frecuencia en los **extremos o bordes** de los rangos de entrada que en el "centro" de los mismos.
- **Cómo se aplica:** En lugar de seleccionar cualquier valor de una clase de equivalencia, se eligen específicamente los valores en los límites.
 - *Ejemplo:* Si una entrada acepta un rango entre **a** y **b**, se deben probar los valores **a**, **b**, y valores inmediatamente fuera del rango como **a-1** y **b+1**.
- **Alcance:** No solo se aplica a las condiciones de entrada, sino que también se obtienen casos de prueba para el **campo de salida** y las estructuras de datos internas.

31. Explicar los 4 tipos de mantenimiento de software / Describir los 4 tipos de mantenimiento de software.

El mantenimiento es la fase de evolución del sistema que ocurre tras su entrega al cliente. Existen cuatro categorías principales:

- **Correctivo:** Es una acción reactiva que se realiza cuando se detecta un fallo o error en el software ya operativo. Su objetivo es diagnosticar la causa, corregir el problema y restablecer el funcionamiento óptimo lo más rápido posible.
- **Adaptativo:** Consiste en modificar el software para que interaccione correctamente con un entorno cambiante. Esto incluye adaptaciones ante nuevos sistemas operativos, cambios en el hardware, almacenamiento en la nube o nuevas reglas y políticas de negocio.
- **Perfectivo:** Se realiza para mejorar la eficiencia, el rendimiento o la experiencia del usuario. Su objetivo es añadir nuevas funcionalidades en respuesta a sugerencias de los clientes o eliminar características que han dejado de ser útiles para mantener el software relevante.
- **Preventivo:** Conjunto de actividades planificadas regularmente para evitar posibles fallos futuros. Incluye tareas como actualizaciones de seguridad, instalación de cortafuegos, prevención de malware y configuración de puntos de restauración del sistema.

37. Explicar los 4 tipos de prueba (Estrategias) / Nombrar las 4 estrategias de prueba del software (no caja blanca o negra) / Indique los distintos tipos de prueba de las estrategias de prueba.

Dentro de una estrategia sistemática de pruebas, se distinguen cuatro etapas o tipos fundamentales que progresan desde "lo pequeño" hacia "lo grande":

- **Prueba de unidad:** Se centra en verificar el correcto funcionamiento de cada componente o módulo individualmente, examinando sus estructuras de datos locales y condiciones límite.
- **Prueba de integración:** Toma los componentes que ya pasaron las pruebas de unidad y los combina según el diseño arquitectónico para verificar que trabajen juntos correctamente y que la información fluya sin errores entre ellos.
- **Prueba de validación:** Asegura que el software construido cumple con los requisitos funcionales y no funcionales establecidos originalmente con el cliente, confirmando que el producto es el "correcto".
- **Prueba del sistema:** Verifica que todos los elementos (software, hardware, personas) encajen de forma adecuada y que el sistema completo realice las funciones apropiadas en su entorno real.

38. Explicar las 4 organizaciones de equipo de trabajo (Paradigmas).

La estructura de un equipo de desarrollo puede seguir diferentes paradigmas organizacionales según el nivel de control e innovación requerido:

- **Paradigma cerrado:** Sigue una jerarquía de autoridad tradicional con comunicación vertical entre el jefe y los miembros. Es eficiente para proyectos similares a otros ya realizados, pero es el menos innovador.
- **Paradigma abierto:** Se basa en el trabajo colaborativo, la gran comunicación y la toma de decisiones por consenso (democrático). Es ideal para resolver problemas complejos, aunque puede no ser tan eficaz en el desempeño ordenado como otros modelos.
- **Paradigma síncrono:** Utiliza la política de "divide y vencerás", organizando a los miembros para trabajar en partes aisladas del problema con poca comunicación activa entre subgrupos. Es adecuado para problemas de gran tamaño y complejidad, pero requiere cuidado para que la falta de comunicación no genere productos incoherentes.
- **Paradigma aleatorio:** Es una estructura holgada que no requiere un líder definido y depende de la iniciativa individual de los miembros. Destaca en avances tecnológicos e innovación pura, pero puede tener dificultades cuando se requiere un desempeño estrictamente ordenado.

39. Explicar cómo funciona cliente-servidor e indicar los participantes.

39. Explicar cómo se usa una prueba de valores límite.

40. Nombrar y explicar las 3 reglas doradas.

Theo Mandel estableció tres reglas doradas fundamentales para el diseño de interfaces de usuario, enfocadas en mejorar la experiencia y la interacción. Adicionalmente, se considera una "cuarta regla" ligada a los aspectos humanos.

Dar control al usuario

El diseño debe contemplar que el usuario busca un sistema que responda a sus necesidades y le facilite sus tareas diarias. Para cumplir con esto, la interfaz debe:

- Evitar obligar al usuario a realizar acciones innecesarias.
- Proporcionar una interacción flexible.
- Incluir opciones que permitan interrumpir o deshacer acciones.
- Simplificar o depurar la forma de interactuar a medida que aumenta la destreza del usuario con el sistema.
- Ocultar los detalles técnicos internos frente a los usuarios ocasionales.
- Permitir una interacción directa con los objetos presentes en la pantalla.

Reducir la carga de memoria del usuario

El sistema no debe exigirle al usuario que recuerde demasiada información al mismo tiempo. Para alivianar esta carga, se debe:

- Reducir la demanda de la memoria a corto plazo.
- Definir valores por defecto que tengan sentido y utilidad.
- Proveer accesos directos que resulten intuitivos.
- Apoyar el formato visual de la interfaz en metáforas del mundo real.
- Desglosar y mostrar la información de manera progresiva.

Lograr consistencia

Una interfaz consistente permite que el usuario aplique conocimientos previos sin tener que aprender cosas nuevas a cada paso. Esto implica:

- Lograr que la tarea actual del usuario se encuadre en un contexto que tenga significado para él.
- Hacer que el usuario pueda determinar fácilmente de dónde viene y hacia dónde puede ir dentro del sistema.
- Mantener una estética y un funcionamiento consistentes en toda la familia de aplicaciones.
- Utilizar exactamente las mismas reglas de diseño para llevar a cabo interacciones similares.
- Mantener los modelos de uso prácticos a lo largo del tiempo, cambiándolos solo si es absolutamente imprescindible.

La cuarta regla teórica: Sumar Factores Humanos

Considerada teóricamente como la cuarta regla, subraya que el diseño siempre se ve afectado por las características intrínsecas de las personas, como su percepción (visual, auditiva y táctil), su razonamiento y su nivel de capacitación. Al aplicar esta regla se debe entender que:

- Existe una diversidad de usuarios: un usuario casual siempre necesitará ser guiado paso a paso, mientras que un usuario experimentado demandará agilidad.
- El diseño debe contemplar las limitaciones de la memoria humana; por ejemplo, una persona generalmente puede retener un promedio de 5 ± 2 piezas de información a la vez.
- Deben tenerse en cuenta las habilidades y el comportamiento personal de cada individuo al interactuar con el entorno.